

دا كذا مرجع لاي حد داخل مثلا انترفيو او بيحاول يراجع ع OOP مراجعة سريعة

ال OOP مش لغة برمجة هي طريقة تنظيمية حنظم الكود وأقسمه لملفات وأخليه سهل إنه يتعدل عليه وسهل أطور فيه وسهل أتحكم في البروجيكت لما الكود يكبر

العلاقة بين Class \$ object

Class ==> Logical Representation ↓

يعني مفهوم في دماغنا وبس يعني مقدرش اتعامل معاها يعني فكرة أو قالب بنوصف فيه شكل الحاجة وسلوكها لكن ما نقدرش نتعامل معاه مباشرة أو نستخدمه إلا لما Create منه (Object)

- تخيل إنك مهندس معماري وعملت رسم تصميم لبنت : التصميم ده هو ال Class
- مش هتقدر تعيش جواه، ولا تفتحه، ولا تبني عليه حاجة — لأنه مجرد تصميم على الورق.
- لكن لما تيجي تبني البنت فعليًا من التصميم ده، يبقى ده هو ال Object

هو نموذج (Blueprint) يُستخدم لإنشاء كائنات (Objects)

يمكنك التفكير في الكلاس كقالب يحدد الخصائص (Properties) والطرق (Methods) التي ستكون متاحة للكائنات التي يتم إنشاؤها منه

(هنا يعني حاجة قادر اتعامل معاها وقادر ادليها قيم) Object is an instance of a class ==> Object
يعني مش لازم الحاجة المسها علشان اقول عليها اوبجكت يعني مثلا لما اتكلم ع مجموعة كورسات ده تبقا كلاس اما لو دخلت جوه وقولت كورس السي شارب واتكلمت ف كل تفاصيله بيقا ده اوبجكت

البنت فعليًا (حاجة قادر اتعامل معاها) Physical representation ==> Object

(Class \$ Object) مبسوط لفهم

نفترض ان الكلاس هو دا الانسان تمام بيقا الاوبجكت هو انا واحمد وحاتم وعلى وادم كلنا لينا نفس الصفات يعني كلنا لينا شعر ويد وعين ورجل وانف وهكذا بس نختلف ف القيم يعني اى يعني مش كلنا لينا نفس العمر ونفس لو الشعر ونفس الطول الحاجات ده بتختلف من شخص للتاني وهكذا الباقي

abstraction هو عملية إخفاء تفاصيل معينة وإظهار المعلومات الأساسية فقط للمستخدم، او هو وصف الكلاس نفسه

مهمة اوى الحته ده انت هنا بتقرر انى الحاجة ده هتبقا موجوده معاك جوه الكلاس او لا وانت بتعمله designe بنانا على Business requirements (abstraction)



1: **Encapsulation** يعنى بعمل تغليف للبيانات وبحاول احميها وأنظم طريقة الوصول لها عن طريق حاجه اسمها access modifiers , يعنى انت هنا بتتحكم فى visibility يعنى بتقرر اى الى ممكن الوصول اليه والتعديل فيه واى لا .

(OOP)

- Don't Repeat Yourself
- Object is an instance of a class

@ تصحيح الصورة استخدام الـ Getter للوصول إلى البيانات

`Console.WriteLine("Name: " + person.getName())`

`Console.WriteLine("Age: " + person.getAge())`

```
class Person {
    // البيانات الخاصة (Private)
    private String name;
    private int age;

    // دالة للوصول إلى قيمة الاسم (Getter)
    public String getName() {
        return name;
    }

    // دالة لتعيين قيمة الاسم (Setter)
    public void setName(String name) {
        this.name = name;
    }

    // دالة للوصول إلى قيمة العمر (Getter)
    public int getAge() {
        return age;
    }

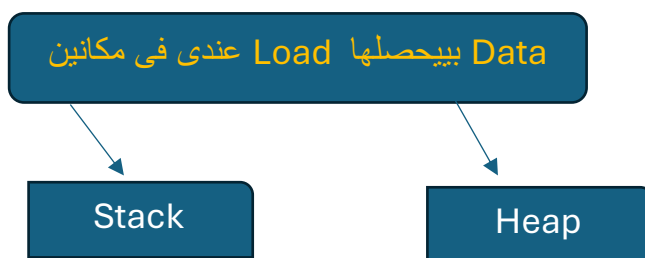
    // دالة لتعيين قيمة العمر (Setter)
    public void setAge(int age) {
        if (age > 0) { // التأكد من أن العمر أكبر من صفر
            this.age = age;
        } else {
            System.out.println("العمر يجب أن يكون أكبر من صفر");
        }
    }
}

public class Main {
    public static void main(String[] args) {
        // إنشاء كائن من نوع Person
        Person person = new Person();

        // تعيين البيانات باستخدام الـ Setter
        person.setName("أحمد");
        person.setAge(25);

        // الوصول إلى البيانات باستخدام الـ Getter
        Console.WriteLine("Name: " + person.Name);
        Console.WriteLine("Age: " + person.Age);
    }
}
```

ده كله ع بعضه اسمه اوبجكت `Car c = New Car ()`



1- **Stack** دا مكان فى memory يحتفظ بالمتغيرات الـ (Local Variables) التي يتم إنشاؤها عند استدعاء دالة.

@ اي Local Variables بيتحتطى فى Stack

خلى ف بالك variables اللى بيتحتط ف Stack بيفضل عايش طول ما Faunction بيحصلها Execution واول ما Faunction ده تخلص الداتا اللى موجوده فى Stack بتتحدف من memory

سوء بقا كان variables (References او value type)

كيف يعمل هذا الكود؟

1. فى البداية، يتم تخصيص مساحة فى الذاكرة للمتغير x فى الـ

Stack داخل دالة Main()

2. عندما يتم استدعاء الدالة MyFunction()، يتم تخصيص مساحة جديدة فى الـ Stack لهذه الدالة، ويتم تخزين المتغير y فيها.

3. بعد أن تنتهي الدالة MyFunction()، يتم إزالة الـ Stack الخاص بها من الذاكرة، وبالتالي يتم التخلص من المتغير y

4. عندما تعود الدالة Main() للـ Execution بعد استدعاء MyFunction()، يظل المتغير x فى الذاكرة لأنه موجود فى الـ Stack الخاص بدالة Main()

```
using System;
class Program
{
    static void Main()
    {
        int x = 10; // متغير محلي
        Console.WriteLine("قبل استدعاء الدالة: " + x);
        MyFunction();
        Console.WriteLine("بعد استدعاء الدالة: " + x);
    }

    static void MyFunction()
    {
        int y = 20; // متغير محلي داخل الدالة
        Console.WriteLine("داخل الدالة: " + y);
    }
}
```

@ Car c خلى ف بالك هنا C دا اسمه References من Class Car يعنى **مش** بيشيل اى قيم فهنا C ده موجود **مش** علشان يشيل اوبجكت من Class Car ده موجود علشان يشيل عنوان الاوبجكت يعنى الخلاصة هنا References مش بيشيل اى قيم ده بيشيل عنوان الاوبجكت فى memory

Car c = New Car ()

New Car () هنا بقا انا كريت اوبجكت من الكلاس Car In Memory

local variables

يمكن يكون value type or References ومش بعرف استخدمه غير لما اعمله initialization يعنى اديله قيمة ; int num = 5

لو جيت هنا مثلا

`int num;`

`int x = num` هنا هيظهر الخطا وبيقولك انك لازم تعمله **initialization** سوء بقا الكلام ع **References** او **value type** اللي هو `int num = 3 ;`

`Car c ;`

`C.Price = 300 ;` نفس الكلام هنا هيظهر الخطا وبيقولك انك لازم تعمله **initialization**

اللي هو `Car c = New Car()`

معلومة ع الماشي اى حاجة **value type** مينفعش اديها قيمة **Null** مثال `Int num = Null ;` هنا خطا ع طول اما **References** فعادى اديها قيمة **Null** مثال `Car c = Null ;`

خلي ف بالك انا مقدرش اعمل اى اوبجكت من غير ما اكتب **New**

`Car c = new car()`

`Car c2 = c`

هنا انا ساويت **References** ب **References** يعنى انا هنا الاتنين **References** بيشاورا على نفس الاوبجكت

=====

What you understand by Value types and Reference types in ?

(تخزين القيمة مباشرة في (**Stack**) value of data (**primitive**) (1)

(int , short , long , double , char , Structure , Enum , bool)

(يعنى تخزين منظم ببيقا فوق بعض)

عند نسخ قيمة من متغير إلى آخر، يتم نسخ القيمة بأكملها. لكن & لو حصل اى تغيير لا يحدث شى لانى غيرت فى القيمة وليس المكان الاصلى اللي هو العنوان بتاعه فى الذاكرة اللي هو بنقول عليه **Reference**

تخزن مباشرة في الذاكرة .



في المثال ده، المتغير **y** بيكون نسخة من **x** يعني لما تغير **x** بعد كده، **y** هتظل زي ما هي، مش هتتأثر

Advantages

A value type does not need extra garbage collection.

عندما يتم إنشاء متغير من نوع قيمة (مثل **int**، **bool**، **double**)، وما إلى ذلك، يتم تخزين قيمته مباشرة في مكانها في الذاكرة. عندما ينتهي نطاق هذا المتغير (على سبيل المثال، عندما تنتهي دالة تعريف المتغير)، يتم تحرير المساحة التي كانت تستخدمها هذه القيمة بشكل تلقائي.

*يعنى اى **int x = 11** ف المتغير والقيمة هيتخزنوا فى **stack**

```
using System;

class Program
{
    static void Main()
    {
        int x = 10; // من نوع قيمة x المتغير
        int y = x;  // مش هتتأثر y، يعني لو غيّرنا x، هي نسخة من y هنا
        x = 20;     // x تغيير قيمة

        Console.WriteLine("x: " + x); // طببع 20
        Console.WriteLine("y: " + y); // طببع 10 (فضلت زي ما هي y هتتأثر)
    }
}
```

(2) non primitive (reference of data)

(array and string class delegate)

@ Reference Type بيتم تخزينه ف مكانين ف الذاكرة **stack** and **heap** حيث يتم تخزين Reference Type في **Stack** ويتم تخزين **object** في **Heap**

❓ ال **Reference** نفسه (المؤشر اللي بيشير للكائن) بيتخزن في ال **Stack**

❓ لكن ال **Object** اللي ال **Reference** بيشير ليه، بيتخزن في ال **Heap**

عندما تقوم بنسخ نوع **Reference** من متغير إلى آخر، فإنك كذا بتنسخ **Reference** وليس البيانات الفعلية. ثم يشير كلا المتغيرين إلى نفس البيانات الموجودة في الذاكرة

لو حصل اى تغير سيتأثر لاني غيرت فى المكان الاصلى اللي هو العنوان بتاعه فى الذاكرة اللي هو بنقول عليه المرجعيه

يعنى اى **string x = "Ali"** فالمتغير **x** (**identifier**) هيتخزن فى **Stack** والقيمة بتتخزن فى **Heap**



مثال

`String[] friends = New string[5]`

اسمه (identifier) `Friends`====>

ده كدا متخزن فى `Stack`

ده كدا متخزن فى `Heap` `[5]`====>

```
using System;

class Car
{
    public string Model { get; set; }
}

class Program
{
    static void Main()
    {
        Car car1 = new Car(); // من نوع إشارة car1 المتغير
        car1.Model = "Toyota"; // إسناد قيمة للخاصية Model

        Car car2 = car1; // يشير ليه car1 يشير لنفس الكائن اللي هنا car2
        car1.Model = "Honda"; // تغيير قيمة الخاصية Model من car1

        Console.WriteLine("car1 Model: " + car1.Model); // طباعة "Honda"
        Console.WriteLine("car2 Model: " + car2.Model); // طباعة "Honda" (لأنهم يشيرون لنفس الكائن)
    }
}
```

this دى تعويض عن الاوبجكت داخل الكلاس بحث توضع مع المتغير مكان الاوبجكت داخل الكلاس

عندما يكون لديك تعارض في أسماء المتغيرات

This بتمثل عندى `Reference` ببشاور على `Carrunt` اوبجكت

This خلى ف بالك هى `hidin parameter` وليس `keyword`

Hidden Parameter يعنى باراميتير فعلي , بيتبع أوتوماتيك, زي `this` في `instance methods` & `extension methods` إنت ما بتكتبوش أثناء الاستدعاء

خلى بالك لا يورث **Constractor**

1- `it is special method` السبب

2- `class` اسمها نفس اسم الكلاس

3- `return type` ملهاش

4- يتم استدعاءه اول ما بكرت اوبجكت من الكلاس اوتوماتيكلى بدون اى تدخل منى

`Car c = new Car();`

ده كده معناه انى بكرت اوبجكت -----> `new Car();`

ده كدا يعنى **Constractor** -----> `Car();`



خلى ف بالك انت مينفعش تعمل اوبجكت من اى كلاس اللى بدلالة **Constructor** طب لو انت ف الكلاس معملتش **Constructor** الكومبيلر بيعمل **Default Constructor** زى كدا

```
public car() {}
```

فاضى ملهوش **body** طب بيعمل اى مبيعلمش اى حاجة بس علشان يمشى ع القاعده اللى بتقول انك متقدرش تكرت اى اوبجكت من غير ما تعمل **Constructor**

- فى حالة انى انت عايز تعمل **class** بدون اى **constructor** اسم الكلاس وليكن (**person**)

هتستخدم دا عادى مش هيطلع اى **error** `person p = new person();`

- ف حالة انك عايز تعمل **constructor** بياخد **pramter** اللى هو دا

```
public numbar1 ( int x1, int x2 , string name ) {}
```

- مش هتعرف تستخدم الطريقة دى `person p = new person();`

الا اذا عملته **Constructor** <----- `public numbar1 () {}`

فوائد **Constructor**

Constructor استخدامه ك **Logec** علشان اتحكم فى اراى الناس تعمل **Creation** للاوبجكت من الكلاس بتاعى

على مستوى **Syntax** بستخدمه علشان اعمل **initialization** لبعض **membar** اللى موجوده فى الكلاس بتاعى

```
Car c = new Car (300) ;
```

خلى بالك هنا وجود **Keyword** اللى هى **New** هو اللى سبب ال **Creation** بتاع الاوبجكت وبعد ما تكرت الاوبجكت بتناده على **Constructor** اللى هو <----- `Car (300)`

<> انواع **Constructor**

- ❖ **Default Constructor**
- ❖ **Private Constructor** we can't create object from them
- ❖ **Parameterized Constructor** (اللى بياخد منك **Paramete**)
- ❖ **Copy Constructor** تستخدم لنسخ القيم من كائن إلى آخر
- ❖ **Static Constructor** لا يمكن استدعاؤه إلا مرة واحدة



هترمي Exception لأن الـ Logic
بيمنع إنشاء أوبجكت بفلوس سالبة.

```
public class BankAccount
{
    public string Owner { get; }
    public decimal Balance { get; private set; }

    public BankAccount(string owner, decimal initialAmount)
    {
        // Logic → إنفورميشن لتواعد معينة
        if (initialAmount < 0)
            throw new ArgumentException("Initial amount can't be negative");

        if (string.IsNullOrEmpty(owner))
            throw new ArgumentException("Owner name is required");

        // Initialization → تهيئة الأعضاء
        Owner = owner;
        Balance = initialAmount;
    }
}

var acc = new BankAccount("Ahmed", -500);  // لو حد حاول يعمل
```

دول مثالين لفهم فوائد
الكونستركٲور كالوجيك وكاسنٲكس

```
public class User
{
    public string Name { get; }

    public User(string name)
    {
        if (string.IsNullOrEmpty(name))
        {
            throw new ArgumentException("Name cannot be empty");
        }

        Name = name;
    }
}

var u = new User("");  // لو حد حاول يعمل هيرمي اكسبشن لانه بيعت الاسم فاضي

+++++

public class Car
{
    public string Model { get; }
    public int Year { get; }
    public int Speed { get; private set; }

    public Car(string model, int year)
    {
        Model = model;    // initialization
        Year = year;      // initialization
        Speed = 0;        // default initialization
    }
}
```



<< Inheritance >>

هي عملية إنشاء كلاس جديد بناءً على كلاس سابق، فتمكننا من إعادة استخدام أكواد مكتوبة سابقاً، وتعديل بعضها أو إضافة خصائص جديدة بدل من إعادة كتابتها كل مرة، وتسمى الفئة **Class** التي نرث منها بالأب **Parent** والتي تُورثُ إليها بالابن **Son**.

<< انواع Inheritance >>

1) Single Level Inheritance

حيث يمكن **Derived Class** أن يرث من **Parent Class** واحد فقط. هذا هو النوع الأكثر شيوعاً من الميراث المستخدم في **OOP** (Single Level Inheritance)

```
public class Animal
{
    public void Eat() { Console.WriteLine("Animal is eating."); }
}

public class Dog : Animal
{
    public void Bark() { Console.WriteLine("Dog is barking."); }
}
```

2) Multi Level Inheritance

derived class ترث من **Parent class** وتصبح **derived class** نفسها **Parent class** لـ **derived class** أخرى.

```
public class Animal
{
    public void Eat() { Console.WriteLine("Animal is eating."); }
}

public class Dog : Animal
{
    public void Bark() { Console.WriteLine("Dog is barking."); }
}

public class Puppy : Dog
{
    public void Play() { Console.WriteLine("Puppy is playing."); }
}
```



3) multiple inheritance

A derived class that inherits from two or more base classes.

يمكن أن يرث الفصل من أكثر من فئة قاعدة واحدة هذا النهج يسمى الميراث المتعدد **Interface**

```
public interface IAnimal
{
    void Eat();
}

public interface IDomestic
{
    void Play();
}

public class Dog : IAnimal, IDomestic
{
    public void Eat() { Console.WriteLine("Dog is eating."); }
    public void Play() { Console.WriteLine("Dog is playing."); }
}
```

Why INHERITANCE ?

 **REUSABILITY** قابل لإعادة الاستخدام

الوراثة تسمح لك بإعادة استخدام الكود الموجود في
(Base Classes) داخل (Derived Class)

دون الحاجة لكتابة نفس الكود مرة أخرى
هذا يُقلل من التكرار في الكود ويسهل عملية التوسع في
المستقبل

لديك **Class Animal** تحتوي على دالة **Eat()** يمكن لأي
class مثل **Dog** أو **Cat** أن تستخدم نفس الدالة بدون
الحاجة لإعادة كتابتها.

```
public class Animal
{
    public void Eat()
    {
        Console.WriteLine("Eating ... ");
    }
}

public class Dog : Animal
{
    public void Bark()
    {
        Console.WriteLine("Barking ... ");
    }
}

public class Cat : Animal
{
    public void Meow()
    {
        Console.WriteLine("Meowing ... ");
    }
}
```

MAINTAINABILITY قابل الصيانة

* الوراثة تجعل الكود أسهل في الصيانة، لأنك لا تحتاج إلى تعديل الكود في كل مكان عند إجراء تحديثات

* عند تحديث الكود في كلاس الاب، سيتم تحديث الكود في جميع كلاسات الابن بشكل تلقائي.
هذا يقلل من المخاطر والأخطاء المحتملة عند تحديث الكود في أماكن متعددة

EXTENSIBILITY قابل التمدد

مثال: يمكنك إضافة سلوكيات جديدة (new behaviors)

أو تخصيص سلوك موجود في كلاس الاب عن طريق تجاوز (Override) الدوال في كلاسات الابن .

هنا **Dog** و **Cat** أضافا سلوكًا مخصصًا للدالة **MakeSound()**، وهذا يُظهر كيف يمكنك توسيع سلوك الفئات المشتقة بما يتناسب مع احتياجاتك.

```
public class Animal
{
    public virtual void MakeSound()
    {
        Console.WriteLine("Animal makes a sound.");
    }
}

public class Dog : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Dog barks.");
    }
}

public class Cat : Animal
{
    public override void MakeSound()
    {
        Console.WriteLine("Cat meows.");
    }
}
```

بص يسيد اى Date type or Class or Enum or Struct

كلهم بيورثوا من حاجة اسمها object دا كدا يعتبر parent لاي datatype عندك

Q1: Can Multiple Inheritance implemented in C# ?

In C#, derived classes can inherit from one base class only. If you want to inherit from multiple base classes, use interface.



Q2: What is inheritance? Does C# support multiple inheritance?

Inheritance means get some of the properties from a Parent class.

عرفت فوراً إن **C#** بتمنع الـ **Multiple Inheritance** للـ **Classes** ، وقدمت الحل البديل وهو الـ **Interfaces** . توضيح إضافي (للمعلومة): السبب الرئيسي لمنع الوراثة المتعددة في الكلاسات هو "مشكلة الماسة" (**The Diamond Problem**) ، وهي إن لو ورثت من كلاسيين والاثنتين فيهم ميثود بنفس الاسم، الكود هيجتار ينفذ مين فيهم؟ عشان كذا الـ **Interface** أضمن لأنه مجرد "عقد" وإننت اللي بتكتب التنفيذ بنفسك

Q4: What are Property Accessors?

ما هي الـ **Property** في **C#** ؟

الـ **Property** في **C#** هي طريقة للوصول إلى البيانات داخل (**Objects**) وتحتوي عادةً على دوال (**get**) و (**set**)

ما هي الـ **Accessors** ؟ الـ **Accessors** هم عبارة عن دوال (**Get**) و (**Set**).

في **C#** ، هناك نوعين من الـ **Accessors**:

1. **Get**: دالة تستخدم علشان اقرا قيمة الـ **Property**

2. **Set**: دالة تستخدم لتعيين وتعديل القيم للـ **Property**

مثال بسيط

عايز نتحكم فيها **Name** ده فيه خاصية اسمها **object** يمثل شخص، وداخل **object** أنت لو عندك كيف نستخدمها؟ بشكل معين

```
class Person
{
    private string name; // هذا المتغير الخاص الذي سيخزن الاسم

    // هي التي ستتفاعل مع المستخدم "Name" الخاصية
    public string Name
    {
        get
        {
            return name; // التي تقرأ قيمة الاسم get هذه هي دالة الـ
        }
        set
        {
            if (!string.IsNullOrEmpty(value)) // هنا نتحقق إذا كان الاسم مش فارغ
            {
                name = value; // التي تعين القيمة set هذه هي دالة الـ
            }
        }
    }
}
```

```
class Program
{
    static void Main()
    {
        Person person = new Person(); // إنشاء كائن من نوع Person
        person.Name = "Ahmed"; // لكتابة القيمة set نستخدم الـ
        Console.WriteLine(person.Name); // لقراءة القيمة get نستخدم الـ
    }
}
```



ليه بنستخدم الـ **Properties** في **C#** بدل المتغيرات العادية؟
لو إنت لسه بتبدأ في **C#** ، أكيد سألت نفسك: ليه أوجع دماغي وأعمل **get** و **Set** طالما ممكن أعمل متغير **public** و خلاص؟

الفرق باختصار هو "الأمان والتحكم"
تخيل إن عندك كلاس **User** وفيه متغير اسمه **Age** لو خليته متغير عادي
(Field) : **public int Age;** أي حد يقدر يكتب : **user.Age = -50;** ..وطبعاً مفيش سن بالسالب! هنا السيستم بتاعك هيضرب.

هنا بيجي دور الـ **Property** كـ "حارس بوابة" الـ **Property** بتديك قوتين خارقتين:
1) التحكم (**Validation**) : تقدر تمنع أي قيم غلط تدخل

```
private int _age;
public int Age {
    get => _age;
    set {
        if (value > 0) _age = value; // مسموح فقط بالقيم المنطقية
        else
            Console.WriteLine("Invalid Age!");
    }
}
```

2) تحديد الصلاحيات (**Access Control**) : ممكن تخلي المتغير "للقراءة فقط" (**Read-only**)
بإنيك تعمل **get** بس، فمحدث يقدر يعدل القيمة من بره الكلاس.

3) الـ **Auto-Implemented Properties** : لو مش محدد شروط معينة، **C#** بيوفرلك
شكل شيك وسريع بيعملك كل حاجة
public string Name { get; set; }

الخلاصة: المتغيرات (**Fields**) هي المخازن اللي بنشيل فيها البيانات، أما الـ **Properties**
هي الأبواب اللي بتحمي المخازن دي. استخدامك ليها بيخلي كودك احترافي، مرن، وصعب
إنه يـ "**Crash**" بسبب بيانات غلط



Polymorphism

مفهوم **Polymorphism** مع مثال:

1. بدون **Polymorphism** دالة عادية بدون **Virtual** و **Override**

الشرح: في هذا المثال، لم نستخدم **virtual** أو **override**، لذلك عندما نستدعي **Move()**، سيتم استدعاء دالة **Move()** الخاصة بـ **Creature** وليس الخاصة بـ **Human**، حتى لو كان الكائن هو **Human** هذا ليس حقيقي.

هنا هينفذ بتاع **Creature** لان نوع **Reference** بتاعى **Creature**

```
class Creature
{
    public void Move()
    {
        Console.WriteLine("Creature");
    }
}

class Human : Creature
{
    public void Move()
    {
        Console.WriteLine("Human");
    }
}

class Program
{
    static void Main()
    {
        Creature c = new Human();
        c.Move(); // سيتم تنفيذ "Creature"
    }
}
```

2. مع **Polymorphism** باستخدام **Virtual** و **Override**

- هنا هينفذ بتاع **Human** عامل **Override** فهينفذ بتاعته الخاصه بيه

\$ يعنى إيه **Polymorphism** ؟

كلمة **Polymorphism** معناها تعدد الأشكال"، والفكرة الأساسية فيه هي:

ان الكود يبقى عنده القدرة يتغير حسب السياق أو الموقف اللي هو فيه."

بمعنى إنك ممكن تستعمل نفس الكود أو نفس الدالة، لكن بتتصرف بطريقة مختلفة بناءً على الظروف.

```
class Creature
{
    public virtual void Move() // virtual تعريف الدالة كـ
    {
        Console.WriteLine("Creature");
    }
}

class Human : Creature
{
    public override void Move() // override إعادة تعريف الدالة باستخدام
    {
        Console.WriteLine("Human");
    }
}

class Program
{
    static void Main()
    {
        Creature c = new Human();
        c.Move(); // سيتم تنفيذ "Human"
    }
}
```

اشكال ال Polymorphism :

1. Compile-Time Polymorphism:(وقت الترجمة)

بيظهر أثناء كتابة الكود وقبل ما يشتغل في وقت ال Compile ال Overloading هو الشكل اللي بيانتمى للنوع ده.

2. Run-Time Polymorphism:(وقت التشغيل)

بيظهر وقت ما الكود بيشتغل في ال Runtime ال Overriding هو الشكل اللي بيانتمى للنوع ده.

ايه Polymorphism .. هو عبارة عن رسالة ببعثها لاكثر من اوبجكت وكل منهم بيتترجمها زي مهو عايز

يعنى Polymorphism هي كلمه مكونه من جزئين "morphs" + "poly" ال poly يعني متعدد و morphs يعني اشكال

لو جماعهم علي بعض يعني متعدد الاشكال

(مثال بسيط)

يعني ايه الكلام دا .. يعني مثلا على سبيل المثال لو عندي Parent class اسمها Animal وعملت Classes بت Inherit منها .. اسمهم Chicken, Elephant, Frog, Snake وعملت فنكشن في ال Animal class وسمتها .. walk

.. لان أي حيوان بيمشي

هاجي اترجمها في ال class اللي اسمها chicken .. واقول بتمشي على رجلي ن
و هترجمها في ال class اللي اسمها Elephant .. واقول بتمشي على اربع
و هترجمها في ال class اللي اسمها Frog .. واقول بتن ط
و هترجمها في ال class اللي اسمها Snake .. واقول بتزح ف
وهكذا

بيقا هو عبارة عن فانكشن في ال Parent class .. ال derived classes بينفذوها بطرق مختلفة



Polymorphism في C# هو مفهوم من مفاهيم ال (OOP) يعني تعدد الأشكال. ويتيح لنا استخدام نفس الدالة في أكثر من Object لكن كل Object يمكن أن يقوم بتنفيذ الدالة بشكل مختلف حسب نوعه.

```
public class Animal
{
    // سيتم تجاوزها في الفئات المشتقة eat() دالة
    public virtual void eat()
    {
        Console.WriteLine("Animal is eating");
    }
}
// الفئة المشتقة (Derived Class)
public class Dog : Animal
{
    // الخاصة بالفئة الأساسية eat() تجاوز دالة
    public override void eat()
    {
        Console.WriteLine("Dog is eating");
    }
}
// فئة أخرى مشتقة (Derived Class)
public class Cat : Animal
{
    // الخاصة بالفئة الأساسية eat() تجاوز دالة
    public override void eat()
    {
        Console.WriteLine("Cat is eating");
    }
}

public class Program
{
    public static void Main(string[] args)
    {
        // إنشاء كائنات من الفئات المختلفة
        Animal myAnimal = new Animal(); // كائن من النوع الأساسي
        Animal myDog = new Dog(); // كائن من النوع المشتق (Dog)
        Animal myCat = new Cat(); // كائن من النوع المشتق (Cat)

        // بناءً على الكائن الفعلي eat() استدعاء الدالة
        myAnimal.eat(); // سيطلع: Animal is eating
        myDog.eat(); // سيطلع: Dog is eating
        myCat.eat(); // سيطلع: Cat is eating
    }
}
```

There are two type of polymorphism in C# ?

1- compile time polymorphism 2- runtime polymorphism

1- compile time polymorphism

- method overloading and operator overloading and static binding or early binding



2- Runtime polymorphism

- method overriding dynamic binding or late binding

=====

Q16: What is an Abstract Class?

abstraction هي عملية إخفاء تفاصيل التنفيذ (**Implementation**) وإظهار الوظائف الأساسية فقط للمستخدم، أو هو وصف الكلاس نفسه

عبارة عن **Class** مينفعش اكرت منه **Object** وال **Methods** بتبقا **Abstract** و **Methods** عادية

هنا يستطيع أي كلاس التورث منه ---> **\$ Abstract class**

ومينفعش تعرف الدالة أو تعملها **implementaion** ف الكلاس اللي هو **\$ Abstract**

بس بتعملها **implementaion** ف الكلاس اللي هو **Child**

#استخدامها: لو عايز تحدد أساس مشترك لكلاسات ثانية

معنى "تحديد أساس مشترك:"

المقصود هنا هو أنك لو عندك أكثر من كلاس بيشاركوا في **functions** أو **behaviors** مشتركة، بدل ما تكرر الكود في كل كلاس، هتكتب الأساس المشترك في **Abstract class** واحد، وبعد كده الكلاسات الثانية هترث من الكلاس ده وتنفذ الوظائف المشتركة مع إضافة تفاصيلها الخاص

ما الفرق بين **Abstract class** و **Sealed class**

1- **Abstract class** ---> هنا يستطيع أي كلاس التورث منه

2- **Sealed class** ----> هنا لا يستطيع أي كلاس التورث منه

استخدامها: لما تكون عايز تمنع الكلاس من التوسع ايه (لا يمكن وراثته الكلاس) أو التعديل عليه **Sealed Class**

✓ الفرق السريع بين **Abstraction** و **Encapsulation** عشان متلخبطش

Abstraction → إخفاء التنفيذ إنت بتستخدم الـ **Method**، لكن ما تعرفش الكود اللي جواها.

Encapsulation → يعني إخفاء البيانات (**Fields**) وجعل التعامل معها يتم فقط من خلال **Methods** أو **Properties**



Abstraction

```
abstract class Animal
{
    public abstract void Speak(); // مفيش تنفيذ ✓
}

class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Dog barks");
    }
}
```

Encapsulation

```
class Person
{
    private int age; // الداتا مخفية ✓

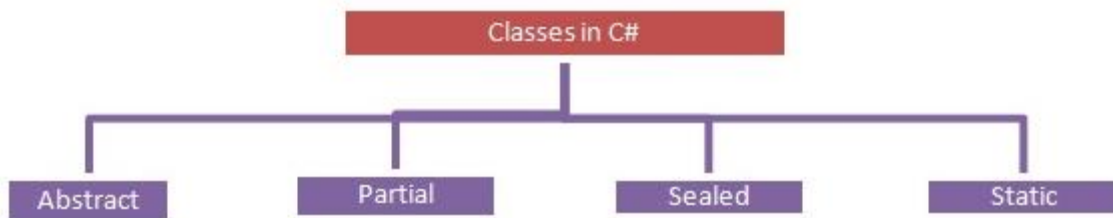
    public void SetAge(int x)
    {
        age = x;
    }

    public int GetAge()
    {
        return age;
    }
}
```

هنا:

- `age` <= التعامل معاها بس عن طريق `SetAge` و `GetAge` ده اسمه **Encapsulation**
- إنت بتتده `Speak()` لكن مش شايف الكود اللي جواها ده اسمه **Abstraction**

What are the different types of classes in C#?



Static class : في لغة C# ، تُستخدم `static class` لتعريف `class` لا يمكن إنشاء `objects` منها ولا يمكن أن تُورث جميع `field` في `static class` يجب أن يكونوا `Static`



```

static class MathHelper
{
    public static int Add(int a, int b)
    {
        return a + b;
    }

    public static int Multiply(int a, int b)
    {
        return a * b;
    }
}

class Program
{
    static void Main()
    {
        // يمكننا استدعاء الدوال الثابتة مباشرة دون الحاجة لإنشاء كائن
        Console.WriteLine(MathHelper.Add(5, 10)); // Output: 15
        Console.WriteLine(MathHelper.Multiply(5, 10)); // Output: 50
    }
}

```

:: Abstract method

لا يمكن استخدامه إلا في **abstract class** ، ولا يحتوي على **body**. يتم توفير **body** من خلال **derived class**

لما يكون عايز اعمل دالة (**method**) **abstract** لازم يكون الكلاس **abstract**

ومينفعش تعرف الدالة او تعملها **implementaion** ف الكلاس اللي هو **abstract**

```

// جزء من الكلاس في الملف الأول
public partial class Car
{
    public string Make { get; set; }
    public string Model { get; set; }
}

// جزء من الكلاس في الملف الثاني
public partial class Car
{
    public void DisplayCarInfo()
    {
        Console.WriteLine($"Make: {Make}, Model: {Model}");
    }
}

class Program
{
    static void Main()
    {
        Car car = new Car();
        car.Make = "Toyota";
        car.Model = "Camry";
        car.DisplayCarInfo();
    }
}

```

4. (Partial Class)

يتيح تقسيم الكلاس الواحد إلى عدة ملفات. كل جزء من الكلاس يمكن أن يحتوي على جزء من الكود، مما يساعد في تنظيم الكود بشكل أفضل.

الشرح: الكلاس **Car** تم تقسيمه إلى جزئين. الجزء الأول يحتوي على خصائص **Make** و **Model**، بينما الجزء الثاني يحتوي على طريقة **DisplayCarInfo()**. هذا يساعد في تقسيم الكود إلى ملفات متعددة.



Partial Class : استخدامها: لما يكون الكود طويل وعايز تقسمه عشان يبقى أسهل في الإدارة

access modifiers

(1) **Public** <==== متشاف ف كل حته يعنى فى اى بروجكت يعنى فى اكثر من namespace

لما اعمل **data member** او **method** في اي **class** واخلوها **public**

معناها كل الناس تقدر تنادى عليها .

(2) **Internal** <==== متشاف ف اى مكان داخل البروجكت يعنى خارج البروجكت ميتشافش يعنى داخل namespace بتعته فقط

الدالة أ و المتغير المعرف ك **internal** يمكن الوصول إليه فقط من الكلاسات الموجودة فى نفس البروجكت.

(3) **Private** <==== داخل الكلاس بتعته فقط

الدوال فى الكلاس الداخلى تستطيع التعامل مع المتغيرات الخاصة بالكلاس الخارجى حتى وان كانت **Private**

(1) **Protected** <==== زيه زى **private** بس كل الحكاية انه متشاف ف الكلاس بتاعه الاساسى و **child** فقط و كمان مش بيظهر ف **main**.

حيث ان المتغيرات والدوال المعرفة ك **protected** يمكن الوصول لها فقط من داخل الكلاس الذي المعرفة فيه او من الكلاسات التى تورث منه

Interview Comparison: Interface vs Abstract Class (C#) 🔥 ✓

Abstract Class	Interface	نقطة المقارنة
كلاس أساس يحتوى تنفيذ وجزء مجرد	عقد (Contract) فقط بدون تنفيذ	التعريف
يدعم Single Inheritance فقط ✗	يدعم Multiple Inheritance ✓	الوراثة
Abstract + Methods عادية	كل الميثودز Abstract	الميثودز
يمكن يحتوى ✓	لا يحتوى ✗	Fields
Public, Private, Protected	Public فقط	Access Modifiers
يوجد ✓	لا يوجد ✗	Constructors
مدعومة ✓	غير مدعومة ✗	Static Members
أساس مشترك لكائنات مرتبطة	سلوك مشترك لكائنات غير مرتبطة	الاستخدام

Interface == **Members are alwase public**

Abstract::Members can have any access modifiers

(What is Enum in C#?)

@ The enum keyword is used to declare an enumeration

Enum is a Value Type

عبارة عن مجموعة من Constant Value

او يتم استخدامه لتخزين مجموعة من الثوابت المسماة مثل الموسم والأيام والشهر والحجم وما إلى ذلك (Enum values are fixed).

Some Points About Enum

@ Enums are value types and are created on the stack and not on the heap.

غالبًا ما يتم استخدام enum لتحسين قابلية قراءة الكود وتقليل الأخطاء عند العمل مع القيم الثابتة

default type is int,

وكمان اى حاجة من مشتقات int زي short و long يعنى متنفعش تكون float او string او double

@ Enumerations (enums) make your code much more readable and understandable

```
public enum Gender
{
    male,    // هي 0 male القيمة الافتراضية لـ
    female   // هي 1 female القيمة الافتراضية لـ
}

Console.WriteLine($"{Gender.female} = " + (int)Gender.female);
Console.WriteLine($"{Gender.male} = " + (int)Gender.male);
```

```
enum days
{
    monday,    // 0
    tuesday,   // 1
    wednesday, // 2
    thursday,  // 3
    friday,    // 4
    saturday,  // 5
    sunday     // 6
}

class Program
{
    static void Main(string[] args)
    {
        // طباعة القيم الخاصة بكل يوم
        Console.WriteLine("Value of Monday is " + (int)days.monday);    // 0
        Console.WriteLine("Value of Tuesday is " + (int)days.tuesday); // 1
        Console.WriteLine("Value of Wednesday is " + (int)days.wednesday); // 2
        Console.WriteLine("Value of Thursday is " + (int)days.thursday); // 3
        Console.WriteLine("Value of Friday is " + (int)days.friday);    // 4
        Console.WriteLine("Value of Saturday is " + (int)days.saturday); // 5
        Console.WriteLine("Value of Sunday is " + (int)days.sunday);    // 6
    }
}
```



عمليات Bit ف الكمبيوتر اللي هي 0001 = 1 0010 = 2 0011 = 3

وهذا يعني أن كل قيمة داخل enum و داخل هذا (OR(|) مثل bitwise operations

قيمة بتية فريدة، ويمكن دمجها باستخدام عمليات البت وانت بتعمل عملية OR (|) ما بين اى رقمين اختار الارقام اللي يكون فيها digit واحد فقط زي 2 = 0100 1 = 1000 يعني مينفعش يكون 3 لان digit بتاعها فيه 11 فيه 1 1 بيقاا نعمل ع الارقام اللي زي 1 و 2 و 4 و 8 و 16 و 32 علشان هذه الارقام لما بحولها الى Binery مش بلاقى فيها غير digit واحد بس

لماذا 3؟ لأن العملية البتية OR تجمع القيم باستخدام الأرقام الثنائية. في هذه الحالة :

• 0001 = 1 = x.c (في النظام الثنائي)

• 0010 = 2 = x.d (في النظام الثنائي)

• نتيجة العملية 0010 | 0001 تكون 0011، وهي تعادل 3 في النظام العشري.

```
using System;

enum x { c = 1, d = 2, g = 4 }

class Program
{
    static void Main(string[] args)
    {
        x v = x.c | x.d;
        Console.WriteLine(v); // سيتم طباعة النتيجة "3"
    }
}
```

[Flags]

يستخدم مع القيم المحتملة انها تشيل اكثر من قيمة

لازمها انك لما تنادى ع اثنين string بدل ما يجمعهم يطبعهم زي ما هم زي Green Red



```

using System;

// تحديد الطوابق Enum تعريف الـ
enum floors
{
    floor1, // (0 هي القيمة الافتراضية له هي)
    floor2, // (1 هي القيمة الافتراضية له هي)
    floor3, // (2 هي القيمة الافتراضية له هي)
    floor4, // (3 هي القيمة الافتراضية له هي)
    floor5, // (4 هي القيمة الافتراضية له هي)
    floor6 // (5 هي القيمة الافتراضية له هي)
}

// تعريف كلاس الموظف
class employee
{
    // الخصائص (الأعضاء) في كلاس الموظف
    public int number; // رقم الموظف
    public string name; // اسم الموظف
    public floors floor; // (من نوع enum) الطابق الذي يعمل فيه الموظف

    // دالة لعرض بيانات الموظف
    public void prentDate()
    {
        // طباعة رقم الموظف، اسمه والطابق الذي يعمل فيه
        Console.WriteLine(number + " , " + name + " , " + floor);
    }
}

class Program
{
    static void Main(string[] args)
    {
        // إنشاء كائن من كلاس الموظف
        employee e1 = new employee();

        // تعيين القيم للموظف الأول
        e1.number = 111; // رقم الموظف
        e1.name = "ali"; // اسم الموظف
        e1.floor = floors.floor1; // تعيين الطابق إلى floor1
        e1.prentDate(); // طباعة بيانات الموظف الأول

        // إنشاء كائن آخر من كلاس الموظف
        employee e2 = new employee();

        // تعيين القيم للموظف الثاني
        e2.number = 777; // رقم الموظف
        e2.name = "mohamed"; // اسم الموظف
        e2.floor = floors.floor5; // تعيين الطابق إلى floor5
        e2.prentDate(); // طباعة بيانات الموظف الثاني
    }
}

```

```

using System;

[Flags]
enum color
{
    red = 2, // 0010
    blue = 2, // 0010
    Green = 4, // 0100
}

class Program
{
    static void Main(string[] args)
    {
        // دمج القيم باستخدام OR (|)
        color c1 = color.blue | color.Green;
        Console.WriteLine(c1); // سيتم طباعة "blue, Green"

        // بين القيم AND (&)
        color c2 = color.blue & color.Green;
        Console.WriteLine(c2); // سيتم طباعة "0"
    }
}

```

عند استخدام العامل & بين القيم، يتم تنفيذ عملية الضرب البتي بينها عند استخدام العامل

=====

(OR) | لدمج القيم .

(AND) & لاستخراج القيم المشتركة.

=====

const : يُستخدم لتخزين قيمة ثابتة واحدة .

enum : يُستخدم لتخزين مجموعة من القيم الثابتة المسماة.

كيف تقوم بتحويل قيمة enum إلى قيمة رقمية او كيف يمكنك تحويل enum إلى int؟



لتحويل **enum** إلى **int**، يمكنك استخدام عملية **Cast**

int إلى **enum** ; // تحويل من **Days.Monday** إلى **(int)**

ماذا يحدث إذا قمت بتعيين نفس القيمة لأكثر من عنصر في **enum**؟

السؤال: "ماذا يحدث إذا تم تعيين نفس القيمة لأكثر من عنصر في **enum**؟"

الإجابة:

- عند تعيين نفس القيمة لأكثر من عنصر في **enum**، فإن ذلك لا يتسبب في خطأ ولكن قد يؤدي إلى عدم وضوح الكود.
- في هذه الحالة، سيتشارك العنصرين في نفس القيمة، ويمكن أن يتسبب ذلك في سلوك غير متوقع عند استخدام **enum**.

```
enum Days
{
    Monday = 1,
    Tuesday = 2,
    Wednesday = 2 // نفس القيمة
}
```

(static)

public static float rateOfInterest = 8.8f ; >>>> Static Field

:: Static Field

لا يوجد سوى نسخة واحدة من **Static Field** الذي تم إنشاؤه في **memory**. تتم مشاركتها مع كافة **Objects**

يعني مفيش غير نسخة واحدة ف ال **memory**

* انا ينفع هنادى ع عنصر **static** داخل دالة **non static**



المتغير **id**: هو متغير (**Static**) ، مما يعني أنه مشترك بين جميع **Objects** من نفس (**Class**) وبالتالي، يمكن الوصول إليه من أي مكان في الكود باستخدام **MyClass.id**.

```
class MyClass
{
    // متغير ثابت (static)
    public static int id;

    // دالة غير ثابتة
    public void xx()
    {
        // يمكنك الوصول إلى المتغير الثابت مباشرة هنا
        id = 333; // هذه عملية صحيحة لأن id هو static
    }
}
```

* لكن ميفعش تنادى ع عنصر **non static** داخل دالة **Static**

Static لما بتتخط قبل **Method**

بيكون هدفها انك تقدر تستخدم الدالة بدون **Object**

يعنى بالعربى مشى هينفع خالص نهائى استخدم الاوبجكت وكذلك مع المتغيرات يعنى **static** بتبقا متاحة ف اى مكان

```
using System;

class Person
{
    // دالة ثابتة (static) تطبع "Welcome"
    public static void Print()
    {
        Console.WriteLine("Welcome");
    }
}

class Program
{
    static void Main(string[] args)
    {
        // الفنة Print استدعاء الدالة من Person
        Person.Print();
    }
}
```



(ما معنى Static Method)

Static Method عبارة عن **Method** يمكن الوصول إليها مباشرةً من **Class** الذي يحتويها بدون الحاجة إلى إنشاء **Object** منه. إذاً الفكرة الأساسية من تعريف **Method** كدالة **Static** هي جعل **Method** قابلة للاستدعاء بشكل مباشر من **Class**

في لغة **#C**، الـ **Static Method** هي **Method** يتم تعريفها داخل **Class** مع الكلمة المفتاحية **Static**. وهذا يعني أنه يمكن استدعاء هذه **Method** بدون الحاجة لإنشاء **Object** (instance) من **Class**

(خصائص الـ Static Method)

1. غير مرتبطة بـ **objects** الكلاس: يتم استدعاؤها عبر اسم **Class** مباشرةً وليس عبر **object** من **Class**.

```
using System;

class Person
{
    // متغير ثابت (Static)
    public static string species = "Homo sapiens";

    // متغير غير ثابت (Instance)
    public string name;

    // دالة ثابتة (Static Method)
    public static void PrintSpecies()
    {
        // (Static) يمكننا الوصول إلى الأعضاء الثابتة
        Console.WriteLine("Species: " + species);

        // (Instance members) لا يمكننا الوصول إلى الأعضاء غير الثابتة
        // هذا سيؤدي إلى خطأ لأنه لا يوجد كائن للوصول إليه // Console.WriteLine("Name: " + name);
    }

    // دالة غير ثابتة (Instance Method)
    public void PrintName()
    {
        // (Instance) والأعضاء غير الثابتة (Static) يمكننا الوصول إلى الأعضاء الثابتة
        Console.WriteLine("Name: " + name);
        Console.WriteLine("Species: " + species);
    }
}

class Program
{
    static void Main()
    {
        // استدعاء الدالة الثابتة بدون إنشاء كائن
        Person.PrintSpecies(); // سيتم طباعة: Species: Homo sapiens

        // إنشاء كائن من الفئة Person
        Person person = new Person();
        person.name = "John";

        // استدعاء الدالة غير الثابتة عبر الكائن
        person.PrintName(); // سيتم طباعة: Name: John \n Species: Homo sapiens
    }
}
```



```

using System;

public class Car
{
    // عضو ثابت (Static)
    public static int carCount = 0; // يتم تخصيص هذا المتغير في الذاكرة بمجرد تحميل الكلاس، ويشارك فيه جميع الكائنات.

    // عضو غير ثابت (Instance Member)
    public string model; // هذا المتغير مرتبط بكل كائن (object) من الفئة، ويخزن في الذاكرة عند إنشاء الكائن.

    // دالة ثابتة (Static Method)
    public static void IncrementCarCount()
    {
        // يمكن الوصول إلى الأعضاء الثابتة فقط داخل الدالة الثابتة.
        carCount++; // زيادة عدد السيارات.

        // من داخل دالة ثابتة "model" لا يمكن الوصول إلى الأعضاء غير الثابتة مثل
        // model = "Tesla"; // خطأ لأن model سيؤدي ذلك إلى خطأ لأن
    }

    // دالة غير ثابتة (Instance Method)
    public void SetModel(string carModel)
    {
        // يمكن الوصول إلى الأعضاء غير الثابتة مثل "model".
        model = carModel; // تعيين موديل السيارة (غير ثابت).

        // من داخل دالة غير ثابتة (مثل carCount) يمكن أيضًا الوصول إلى الأعضاء الثابتة.
        carCount++; // زيادة عدد السيارات.
    }

    // دالة ثابتة (Static Method) لعرض إجمالي عدد السيارات
    public static void ShowCarCount()
    {
        Console.WriteLine("Total cars: " + carCount); // طباعة العدد الإجمالي للسيارات.
    }

    // البرنامج الرئيسي (Main)
    public static void Main(string[] args)
    {
        // استدعاء دالة ثابتة (Static Method)
        Car.IncrementCarCount(); // زيادة عدد السيارات.
        Car.ShowCarCount(); // عرض العدد الإجمالي للسيارات.

        // إنشاء كائن من الفئة Car
        Car myCar = new Car(); // عند إنشاء كائن جديد، سيتم تخصيص الذاكرة له.

        // استدعاء دالة غير ثابتة عبر الكائن (Instance Method)
        myCar.SetModel("Tesla"); // تعيين موديل السيارة للكائن myCar.

        // عرض العدد الإجمالي للسيارات عبر الكلاس (Static Method)
        Car.ShowCarCount(); // عرض العدد الإجمالي للسيارات بعد إضافة كائن جديد.
    }
}

```

(Unreachable)

معناه object مفيش reference بيشاور عليه وبالتالي مفيش حد عارف يوصله وبالتالي يقوم

carbage collector يقفشوه



انا لو عندى **Class** وعملت منه اتنين اوبجكت مثال @

ده معناه انى بساوى **Reference** ب **Reference** يعنى هما الاتنين بيشاوروا ع نفس الاوبجكت **e1 = e2**

انا لو عندى **Struct** وعملت منه اتنين اوبجكت مثال @

ده معناه انى باخد القيم اللى فى الاوبجكت التانى بحطه فى الاول **e1 = e2**

```
Int x = new int ();
```

هنا انا معملتش اوبجكت من **int** هنا انا عملت **initialization** من **int** لانه من النوع **value type** فكنت لازم اعمله **initialization** علشان اعرف استخدمه

```
Struct employee { }
```

```
Employee c = new employee ()
```

نفس الكلام هنا معملتش اوبجكت من **Struct** هنا انا عملت **initialization** من **Struct** لانه من النوع **value type** فكنت لازم اعمله **initialization** علشان اعرف استخدمه

Override, New and Virtual keywords

ال **virtual** هي دي الفانكشن اللي بتبقا في ال **Parent class** واللي هتتغير في ال **derived Classes**

"تسمح بأن يتم تعديل أو تغيير سلوك دالة موجودة في **Parent Class** بواسطة **Derived Class**

تسمح للدالة بال **(override)** في **Derived class**



ال **override** :: :

provides a new implementation for an inherited method in a derived class

عندما نقوم بتوريث (**inheritance**) كلاس من كلاس أخرى قد تكون ال **derived class** بحاجة لتعديل أو تغيير سلوك بعض الدوال التي ورثتها من (**parent class**).

هنا تأتي أهمية كلمة **override**.

- (**Parent Class**) قد تحتوي على دالة معينة.
- (**Derived Class**) يمكنها استخدام كلمة **override** لتوفير تنفيذ جديد أو سلوك مختلف لهذه الدالة الموروثة.

ال **New** .. دي اسمها **method hiding** .. يعني مش بغير الفانكشن اللي كانت في ال **Parent** !! لا .. انا بسببها بس بخفيها

=====

(Over Loading)

تعني تعريف أكثر من **method** أو **Constructor** بنفس الاسم لكن مختلفين عن بعض في

(1) **Number of parameters** (مثلاً: دالة تأخذ معاملين وأخرى تأخذ ثلاثة

(2) **Type of parameters** (مثلاً: دالة تأخذ معاملات من نوع **int** وأخرى تأخذ معاملات من نوع **string**)

(3) **Order of parameters** (مثلاً: دالة تأخذ معامل من نوع **int** ثم **string**، وأخرى تأخذ **string** ثم **int**) Signature is different

```
public void Print(int a, string b) { Console.WriteLine(a + " " + b); } // string ثم int دالة تأخذ
public void Print(string a, int b) { Console.WriteLine(a + " " + b); } // int ثم string دالة تأخذ
```

هل يمكن عمل Overloading للطريقة Main في C# ؟

الإجابة: لا يمكن عمل Overloading للدالة Main لأن C# يتطلب أن تكون الدالة (Static) Main وهي نقطة دخول البرنامج، ولا يمكن وجود أكثر من Signature لها.

في C#، عندما تقوم بعمل Overloading للـ Operator == Operator الذي يقوم بالتحقق إذا كان كائنات متساويين)، يجب عليك أيضًا عمل Overloading للـ Operator != Operator الذي يتحقق إذا كان الكائنات غير متساويين). السبب في ذلك هو أن منطق المقارنة بين Objects يجب أن يكون متناسقًا بين Operators وكما أن C# ما بتفترضش تلقائيًا إن != هو عكس ==

إذا قمت بعمل Overloading للمشغل ==، يجب عليك أيضًا عمل Overloading للمشغل != لأنهما يجب أن يكونا متناسقين مع بعضهما البعض بمعنى آخر :

- إذا كانت == تُرجع true (أي أن الكائنات متساويين)، فيجب أن تُرجع != false أي أنها ليسا غير متساويين).
- إذا كانت == تُرجع false (أي أن الكائنات غير متساويين)، فيجب أن تُرجع != true أي أنها غير متساويين).

```
public int sum(int num1, int num2) {  
    return num1 + num2; // تقوم بإرجاع مجموع العددين  
}  
  
public int sum(float num1, int num2) {  
    return (int) num1 + num2; // تحويل العدد العشري إلى عدد صحيح وإرجاع المجموع  
}  
  
public int sum() {  
    return 0; // يمكن تعديلها لإرجاع مجموع قيم أخرى أو قيمة افتراضية  
}
```

```

using System;

public class Money
{
    // المتغير الخاص لتخزين المبلغ المالي
    private decimal amount;

    // الفنتشي الذي يحدد قيمة المبلغ عند إنشاء كائن جديد
    public Money(decimal value)
    {
        this.amount = value;
    }

    // الطريقة التقليدية للوصول إلى قيمة المبلغ (getter and setter)
    public decimal Amount
    {
        get { return amount; } // الحصول على قيمة المبلغ
        set { amount = value; } // تعيين قيمة جديدة للمبلغ
    }

    // (معا Money إضافة كائنين من النوع) مشغل الجمع
    public static Money operator +(Money money1, Money money2)
    {
        var z = money1.amount + money2.amount;
        return new Money(z); // إرجاع كائن جديد يحتوي على المبلغ الناتج من الجمع
    }

    // (من كائن آخر Money طرح كائن من نوع) مشغل الطرح
    public static Money operator -(Money money1, Money money2)
    {
        var z = money1.amount - money2.amount;
        return new Money(z); // إرجاع كائن جديد يحتوي على المبلغ الناتج من الطرح
    }

    // money2 أكبر من المبلغ في money1 التحقق إذا كان المبلغ في) مشغل المقارنة أكبر من
    public static bool operator >(Money money1, Money money2)
    {
        return money1.amount > money2.amount;
    }

    // money2 أصغر من المبلغ في money1 التحقق إذا كان المبلغ في) مشغل المقارنة أصغر من
    public static bool operator <(Money money1, Money money2)
    {
        return money1.amount < money2.amount;
    }

    // money2 يساوي المبلغ في money1 التحقق إذا كان المبلغ في) مشغل المقارنة المساواة
    public static bool operator ==(Money money1, Money money2)
    {
        return money1.amount == money2.amount;
    }

    // money2 لا يساوي المبلغ في money1 التحقق إذا كان المبلغ في) مشغل المقارنة التفاوت
    public static bool operator !=(Money money1, Money money2)
    {
        return money1.amount != money2.amount;
    }

    // مشغل الزيادة (زيادة المبلغ بمقدار 1)
    public static Money operator ++(Money money)
    {
        var ss = money.amount;
        return new Money(++ss); // إرجاع كائن جديد مع القيمة بعد الزيادة
    }

    // مشغل التناقص (تقليل المبلغ بمقدار 1)
    public static Money operator --(Money money)
    {
        var ss = money.amount;
        return new Money(--ss); // إرجاع كائن جديد مع القيمة بعد التقليل
    }

    // Money برنامج تجريبي لاستخدام العمليات المختلفة على كائنات من النوع
    public static void Main()
    {
        // Money إنشاء كائنين من النوع
        Money m1 = new Money(10); // المبلغ الأول هو 10
        Money m2 = new Money(20); // المبلغ الثاني هو 20

        // استخدام مشغل الجمع لجمع المبالغ
        Money m3 = m1 + m2; // 30 = 20 + 10
        Console.WriteLine(m3.Amount); // طباعة المبلغ الناتج (30)

        // استخدام مشغل الزيادة على كائن m1
        Money m4 = ++m1; // زيادة المبلغ بمقدار 1
        Console.WriteLine(m4.Amount); // طباعة المبلغ بعد الزيادة (11)

        // استخدام مشغل التناقص على كائن m1
        Money m5 = --m2; // تقليل المبلغ بمقدار 1
        Console.WriteLine(m5.Amount); // طباعة المبلغ بعد التقليل (19)
    }
}

```

(Override)

Override : تعني أنك تعيد تعريف نفس الدالة (Same name, same signature) التي موجودة في **Parent Class**، بحيث يتغير **behavior** في **Child Class**.

ده يعنى اكثر من **method** بنفس **Same name** و **same signature**

هل يمكن استخدام **Override** في **static methods**؟ لا يمكن استخدام **override** في الـ **static methods** في **C#** الـ **override** يعمل فقط مع **non-static methods** لأن الـ **static methods** لا تنتمي إلى كائنات معينة من **Class**، وإنما تكون مرتبطة بالـ **Class** نفسه.

```
using System;

public class Animal
{
    // 'virtual' في الصف الأب باستخدام الكلمة المفتاحية Speak تعريف طريقة
    public virtual void Speak()
    {
        Console.WriteLine("Animal speaks");
    }
}

public class Dog : Animal
{
    // في الصف الفرعي Speak طريقة (Override) هنا نقوم بإعادة تعريف
    public override void Speak()
    {
        Console.WriteLine("Dog barks");
    }
}

class Program
{
    static void Main(string[] args)
    {
        // إنشاء كائن من الصف الأب
        Animal animal = new Animal();
        animal.Speak(); // ستطبع: "Animal speaks"

        // إنشاء كائن من الصف الفرعي
        Dog dog = new Dog();
        dog.Speak(); // ستطبع: "Dog barks"

        // إنشاء كائن من النوع الأب ولكن الإشارة إلى النوع الفرعي: Polymorphism استخدام الـ
        Animal myAnimal = new Dog();
        myAnimal.Speak(); // ستطبع: "Dog barks"
    }
}
```



لماذا لا يمكن الوصول إلى Static Members عبر this؟

this تشير إلى object الحالي من Class التي يتم العمل بها. وعندما نقول إن static members لا يمكن أن تكون موجودة داخل "this"، فإننا نعني أن Static Members لا يمكن الوصول إليها عبر objects لأن Static Members مرتبطة بال Class نفسها وليست مرتبطة بأي objects فردية.

=====

@ object هو parent class لاى داتا طيب عندى فى السى شارب

Class Employee : Humen

Employee وارث من Humen وال Humen وارث من Parent class اللى اسمه Object

يبقا بشكل غير مباشر Employee وارث من Object

خلى ف بالك وانت بتعمل

Humen h = new Humen

h.ToString ; h.GetType ; h.Equals ;

Equals باستخدامها علشان اقارن اوبجكت باوبجكت او reference ب reference او اعملها

Override واقارن داتا ب داتا

كل هذا function بتخص class parent اللى اسمه object

Q11: How can you prevent a class from overriding in C#?

by using the sealed keyword.



ايه الفرق بين **Const** و **Static** ؟

1 (**Static**) يُستخدم لجعل **Variable** أو **functions** مرتبطة بال **Class** نفسه وليس بال **Object** يمكن استخدام **static** مع **Variables** ، **functions** ، أو حتى **Attributes**.

static method تكون مشتركة بين جميع **Objects** من **Class** نفسه.

يعني أن **Variables** أو **Static Method** (التي تم تعريفها باستخدام **static** تنتمي إلى **Class** نفسه، وليس إلى **Objects** التي يتم إنشاؤها من هذا ال **Class**

2 (**Const**) : يُستخدم لتحديد قيمة ثابتة لا يمكن تغييرها بعد تعريفها .

بمجرد تعريف قيمة كـ **const**، يجب أن تكون ثابتة طوال فترة حياة البرنامج .

قيمة ثابتة لا يمكن تغييرها // `public const double Pi = 3.14159;`

=====

- ✓ Arithmetic Operators اسمها عوامل حسابية $x = v + y$
- ✓ Comparison Operators اسمها عوامل مقارنات $v > y$
- ✓ Boolean Operators

يعني لو اشتغل ع متغيرين `int x = y + z`

Binary operator سوء جمع طرح قسمه مقارنة اكبر من واصغر من اسمه

`int x = y` هنا بشتغل ع متغير واحد فقط هذا يسمى (**Unary operator** احادي)

=====

What is a Destructor in C# and when shall I create one?

هو دالة خاصة يتم استدعاؤها تلقائيًا عندما يتم تدمير **object** بواسطة **Garbage Collector**



خصائص الـ Destructor :

1. لا يمكن استدعاؤه مباشرة:

○ الـ Destructor يتم استدعاؤه تلقائيًا بواسطة (Garbage Collector) ولا يمكنك استدعاؤه يدويًا مثل الطرق العادية.

2. لا يمكن أن يحتوي على Parameters :

○ الـ Destructor في C# لا يمكن أن يحتوي على Parameters ، وبالتالي لا يمكن تمرير أي بيانات عند استدعائه.

يُستدعى في وقت غير معلوم ⚠ (نقطة مهمة ناقصة عندك)
دي نقطة مهمة جدًا:

- أنت لا تعرف متى سيعمل الـ Destructor بالضبط
- لأنه يعتمد على توقيت عمل Garbage Collector
- ممكن يحصل بعد ثواني أو بعد دقائق أو قد لا يحدث قبل غلق البرنامج

الهدف الاساسى من Destructor

ويستخدم الـ Destructor لتحرير الموارد التي لا يمكن إدارتها بشكل تلقائي بواسطة .NET Framework. Garbage Collector يقوم تلقائيًا بتتبع الكائنات التي لم تعد مستخدمة (أي ليس لها refernce) ويحذفها

◆ هل يمكن عمل Overloading للـ Destructor ؟

✗ لا

• لا يقبل Parameters

• لا يمكن كتابة أكثر من Destructor في نفس الكلاس

كثير منّا يستخدم الـ Constructor عشان يبنى obget، بس قليل اللي بيعرف يتعامل صح مع الـ Destructor (أو الـ Finalizer).

ليه الـ Destructor "مرعب" شوية في الـ C#؟ لأنه ببساطة Out of your control !💡

أهم 3 حاجات لازم تعرفها عنه:



- 1) محدث عارف هيشغل إمتى: الـ **Garbage Collector** هو اللي بيقرر يستدعيه لما الـ **memory** تتملي. ممكن يشتغل بعد ثانية، وممكن ميشغلش خالص غير والبرنامج بيقل
- 2) ممنوع الـ **Overloading**: الكلاس ملوش غير **Destructor** واحد بس، ومابياخدش أي **Parameters** ولا **Access Modifiers** (لا **public** ولا **private**).
- 3) تحرير الموارد (**Unmanaged Resources**): استخدامه الأساسي مش تنظيف الـ **memory** العادية (لأن الـ **GC** بيعمل ده لوحده)، لكن استخدامه في قفل الـ **Files** أو الـ **Database Connections** اللي الـ **Framework** مش بيعرف يتعامل معاها لوحده يعني ايه النقطة دي الـ **(GC) Garbage Collector** في الـ **.NET**. ذكي جداً في إدارة الـ **Managed Memory** زي الأوبجكتس اللي بتعملها بـ **new**، لكنه "أعمى" تماماً تجاه الـ **Unmanaged Resources**.

ايه ده اتلعبط ايه هو اصلا الـ **Managed** و الـ **Unmanaged** ؟

- **Managed Resources** : هي أي حاجة الـ **CLR** بيتحكم فيها وبيرميها لما تخلص (مثل **String, List, Array**).
- **Unmanaged Resources** : هي موارد "خارج" سلطة الـ **.NET** ، وعادةً بنفتحها من خلال نظام التشغيل (**OS**).

ليه لازم نتدخل يدويًا؟

بما إن الـ **GC** مش عارف حجم أو أهمية الـ **Unmanaged Resource** ، فلو استنيناه هو اللي ينظف، ممكن تحصل المشاكل دي:

1. **Memory Leaks** : البرنامج يستهلك رامات كتير والـ **GC** مش حاسس إن فيه خطر.
2. **Resource Exhaustion** : لو فتحت 100 ملف ومقفلتهمش، نظام التشغيل ممكن يرفض يفتح الملف الـ 101 حتى لو الرامات فاضية.
3. **Data Corruption** : لو مقفلتش الـ **Database Connection** أو الـ **File Stream** ، ممكن البيانات متتحفظش بشكل كامل.



Q13: What is difference between late binding and early binding in C#?

Early Binding يعرف ايضا ب static binding or compile-time binding

هنا انا بعرف نوع الاوبجكت اثناء Compile time او معنى ثانى

هو ان نوع الكائن (Type) والميثود اللي هتستدعيها بيكونوا معروفين وقت ال Compile وقت كتابة الكود
يعني : الكومبايلر يعرف كل حاجة بدري , يحصل فحص أخطاء بدري , الأداء أسرع

Late Binding: يعرف ايضا ب dynamic binding or runtime binding .

هنا مش بقدر editect نوع الاوبجكت اللي انا شغال معه غير ف runtime, هو ان نوع الكائن
والميثود لا يتم تحديدهم إلا وقت التشغيل (Runtime)

يعني : الكومبايلر لا يعرف الميثود , لا يوجد فحص وقت ال Compile الأخطاء تظهر وقت التشغيل ⚠️

Base

base keyword is used to access fields, constructors and methods of base class.

```
public class Animal
{
    // في الفئة الأساسية المتغير color
    public string color = "white";

    // (virtual) التي يمكن تعديلها في الفئات المشتقة eat دالة
    public virtual void eat()
    {
        Console.WriteLine("eating..."); // طباعة "eating..."
    }
}

public class Dog : Animal
{
    // في الفئة الأساسية color والذي يخزن المتغير (Dog) في الفئة المشتقة color المتغير
    public string color = "black";

    // دالة eat override تم تعديلها باستخدام
    public override void eat()
    {
        base.eat(); // استدعاء دالة eat() من الفئة الأساسية
        Console.WriteLine("eating bread..."); // طباعة "eating bread..."
    }

    // دالة لعرض الألوان من الفئة الأساسية والمشتقة
    public void showColor()
    {
        Console.WriteLine(base.color); // طباعة اللون من الفئة الأساسية "white"
        Console.WriteLine(color); // طباعة اللون من الفئة المشتقة (Dog) وهو "black"
    }

    // بدون معلمات Dog منشئ الفئة
    public Dog() : base()
    {
        // يمكن إضافة تهيئة خاصة هنا إذا لزم الأمر
    }
}
```

✓ جملة ذهبية تحفظها:

عند إنشاء **object** من كلاس **child**، يتم تنفيذ **Constructor** الكلاس **parente** أولاً ثم كلاس **child**

✓ له حصل كده رغم إننا ما كتبناش **base**؟

لأن **C#** تضيف **base()** تلقائياً في الخلفية بشرط: أن يكون في **Base Class Constructor** بدون **Parameters** يعني الكود ده فعلياً بيتترجم كأنه:

`public B() : base() { ... }`

✓ حتى لو أنت ما كتبتهمش

`public C() : base() { ... }`

```
class A
{
    public A()
    {
        Console.WriteLine("Constructor A");
    }
}

class B : A
{
    public B()
    {
        Console.WriteLine("Constructor B");
    }
}

class C : B
{
    public C()
    {
        Console.WriteLine("Constructor C");
    }
}

C obj = new C();           ==> Alrasuelt

Constructor A
Constructor B
Constructor C
```



✓ صيغة قصيرة لحفظها للإنترفيو:

إذا كان للـ **Base Class Constructors** ببارامترات فقط، يجب على الـ **Derived Class** أن يستدعي أحدها باستخدام **base(...)**، وإلا يفشل الـ **Compile**.

```
class Parent
{
    public Parent(string name)
    {
        Console.WriteLine("Parent: " + name);
    }
}

class Child : Parent
{
    public Child() : base("Ahmed") // ✓ إجباري
    {
        Console.WriteLine("Child");
    }
}

Child c = new Child();
Parent: Ahmed
Child
```

النتيجة

```
class Parent
{
    public Parent()
    {
        Console.WriteLine("Parent Default");
    }

    public Parent(string name)
    {
        Console.WriteLine("Parent: " + name);
    }
}

class Child : Parent
{
    public Child() // ✓ اختياري هنا base
    {
        Console.WriteLine("Child");
    }
}
```

لأن فيه Parent() فارغ

C# تلقائيًا تستدعيه:

public Child() : base ()



```

class Parent
{
    public Parent(string name)
    {
        Console.WriteLine(name);
    }
}

class Child : Parent
{
    public Child()
    {
        Console.WriteLine("Child");
    }
}

public Child() : base() // ✗

```

✗ كلاس أب ليس لديه **Constructor** فارغ

👉 هنا **Parent** يحتاج قيمة **name** إجباريًا.

كلاس **Child** بدون **base** غلط

✗ هنا يحصل **Compile Error**

ليه؟ لأن **C#** حاول تلقائيًا يعمل:

`public Child() : base() // ✗`

لكن **Parent** لا يملك **Parent()** فارغ أصلاً.

✓ جملة تحفظها:

this تشير إلى **Curent object** من نوع الكلاس الحالي، بينما **base** تشير إلى الجزء الموروث من الكلاس **parent** داخل نفس الكائن.

```

class Parent
{
    public int Age = 50;
}

class Child : Parent
{
    public int Age = 20;
}

public void Show()
{
    Console.WriteLine(base.Age); // 50 ✓
    Console.WriteLine(this.Age); // 20 ✓
}

```

base.Age

• معناها:

هات **Age** اللي موجود في جزء **Parent** داخل **object Child**

• لذلك = 50

✓ **this.Age**

• معناها:

هات **Age** اللي موجود في جزء **Child** نفسه

• لذلك = 20

`Console.WriteLine(Age)`

✓ لو كتبت بدون **this** ولا **base**:

سيأخذ أقرب تعريف في نفس الكلاس → يعني : 20



هل الـ **String** في **C#** فعلاً **Reference Type** ؟ ولله مبيتصرفش زيهم؟ 😞
كثير منا عارف إن الـ **String** هو **Reference Type** ، يعني المتغير بتاعه (**Identifier**)
بيكون في الـ **Stack** والقيمة الفعلية بتكون في الـ **Heap** لكن لو جربت تعدل قيمة **String** ،
هتلاقي سلوكه غريب شوية ومختلف عن الـ **Classes** العادية!

إيه السر؟ السر في كلمة واحدة : **Immutability** (عدم القابلية للتغيير)
تعالوا نشوف إيه اللي بيحصل في الـ **Memory** : لو كتبت كود زي ده:

```
string name = "Ali";  
name = "Ahmed";
```

- (1) في السطر الأول: الكومبيلر بيحجز مكان في الـ **Heap** فيه كلمة "**Ali**" ، والمتغير **name** في الـ **Stack** بيشيل "عنوان" المكان ده.
- (2) في السطر الثاني: إحنا متخيلين إن البرنامج هيروح لنفس المكان في الـ **Heap** ويمسح "**Ali**" ويكتب "**Ahmed**" .. بس ده مش بيحصل!

اللي بيحصل فعلياً إن الـ **C#** بيعمل **Object** جديد تماماً في الـ **Heap** ويكتب فيه "**Ahmed**" ، ويخلي المتغير **name** يغير اتجاهه (**Pointer**) ويشاور على العنوان الجديد.

طب وكلمة "**Ali**" القديمة؟ بتفضل موجودة في الـ **Heap** لحد ما الـ **Garbage Collector** يلاحظ إن مفيش أي متغير بيشار عليها، فيقوم بمسحها وتفرغ مساحتها.
نصيحة ليك: لو عندك "**Loop**" بتعدل في **String** آلاف المرات، بلاش تستخدم الـ **string** العادي عشان متملاش الـ **Memory Objects** ملهاش لازمة وتتعب الـ **Garbage Collector**.. الأفضل تستخدم **StringBuilder** لأنه مصمم يكون **Mutable** وبيعدل في نفس المكان في الذاكرة

```
String name1 = "Ali";    String name2 = name1;  
  
name1 = "mohamed";  
  
Console.WriteLine(name1); // mohamed  
  
Console.WriteLine(name2); // ali
```

